PROJETO FINAL: ENGENHARIA DE SISTEMAS DISTRIBUÍDOS – 2026.1

Hands-On

Documento de Definição de Temas e Critérios de Avaliação

Equipes de 5 pessoas | 3 Entregas

Junho 2026

Projeto 01: 26/06 | **Projeto 02:** 10/07 | **Projeto 03:** 07/08

Sumário

Sumário 2
1. Visão Geral do Projeto 3
    1.1. Entregas Obrigatórias 3
    1.2. Formato das Apresentações 3
2. Cronograma de Entregas 4
    2.1. Projeto 01 — Definição do Grupo e Tema (26/06/2026) 4
    2.2. Projeto 02 — Documentação Inicial do Projeto (10/07/2026) 4
    2.3. Projeto 03 — Documentação Final do Projeto (07/08/2026) 4
3. Opção de Tema Próprio 5
4. POCs de Microsserviços Sugeridas 6
    4.1. POC 1 — Real-Time (Salas/Rodadas + Sincronia de Evento) 6
        Escopo 6
    4.2. POC 2 — Antifraude Mínimo Viável 7
        Escopo 7
    4.3. POC 3 — Ledger + PIX Sandbox com Idempotência 8
        Escopo 8
    4.4. POC 4— Anti-Platform Throttling 9
        Escopo 9
5. Critérios de Avaliação 10
    Uso de IA — declaração obrigatória 10
6. Tópicos Técnicos por Área 11
    6.1. Segurança 11
        Tópicos do Curso 11
    6.2. Escalabilidade 12
    6.3. Desempenho 12
    6.4. Confiabilidade 13
    6.5. Deployment 13
7. Checklist de Entrega Final 14
8. Observações Finais 15

1. Visão Geral do Projeto

Este documento define os temas, escopos, cronograma e critérios de avaliação para o Projeto de Arquitetura de Software. Cada equipe (5 integrantes) deverá escolher um dos temas propostos — ou propor um tema próprio — e desenvolver uma solução hands-on que demonstre domínio de padrões arquiteturais e boas práticas de engenharia de software.

O projeto é dividido em três entregas progressivas com datas fixas, culminando na entrega final com documentação completa, repositório funcional e videocast da solução.

1.1. Entregas Obrigatórias

Entrega	Descrição
Repositório Git	Código-fonte completo em repositório Git (GitHub/GitLab) com README documentado, instruções de execução e diagrama de arquitetura.
Apresentação	Apresentação de 12 a 15 minutos mostrando a solução proposta: contexto do problema, decisões arquiteturais, demonstração funcional e lições aprendidas.
Documentação	ADRs (Architecture Decision Records), diagrama C4 (pelo menos níveis 1 e 2), trade-offs documentados e plano de testes.

1.2. Formato das Apresentações

- Duração: 12 - 15 (mínimo/máximo rigoroso)
- Todos os 5 integrantes devem participar da apresentação
- Estrutura sugerida: (1) Contexto e problema, (2) Arquitetura proposta e decisões, (3) Demo ao vivo ou gravada, (4) Resultados e métricas, (5) Lições aprendidas
- Pode ser gravado via Google Meet, Zoom, OBS ou ferramenta equivalente
- O link do vídeo (YouTube unlisted ou Google Drive) deve constar no README do repositório

2. Cronograma de Entregas

O projeto segue um cronograma de três entregas obrigatórias com datas fixas. O não cumprimento de uma entrega implica penalização na nota final.

Entrega	Descrição	Data Limite	Peso
Projeto 01	Definição do Grupo e Tema	**26/06/2026**	**10%**
Projeto 02	Documentação Inicial do Projeto	**10/07/2026**	**20%**
Projeto 03	Documentação Final do Projeto	**07/08/2026**	**70%**

2.1. Projeto 01 — Definição do Grupo e Tema (26/06/2026)

- Formação da equipe (5 integrantes) com nomes e contatos
- Escolha do tema: uma das 5 POCs sugeridas OU tema próprio
- Se tema próprio: proposta escrita (1 página) para aprovação do professor
- Criação do repositório Git com README inicial

2.2. Projeto 02 — Documentação Inicial do Projeto (10/07/2026)

- Diagrama C4 (níveis 1 e 2): contexto e containers
- ADRs iniciais: pelo menos 2 decisões arquiteturais documentadas
- Definição do stack tecnológico e justificativas
- Identificação dos padrões arquiteturais que serão aplicados (mín. 3)
- Setup inicial do ambiente (Docker Compose, CI básico)
- Divisão de responsabilidades entre os integrantes
- Apresentação (12–15 min)
-

2.3. Projeto 03 — Documentação Final do Projeto (07/08/2026)

- Repositório completo com código funcional e testes
- Documentação final: ADRs completos, diagrama C4 atualizado, trade-offs
- Plano e resultados de testes (carga, resiliência, integração)
- README final com instruções de execução completas
- Apresentação Final(12–15 min)

3. Opção de Tema Próprio

Cada equipe pode propor um tema próprio, desde que atenda aos seguintes requisitos mínimos:

1. O projeto deve envolver arquitetura de microsserviços ou sistema distribuído
2. Deve abordar pelo menos 3 padrões arquiteturais listados na Seção 6 deste documento
3. Deve cobrir ao menos 2 das áreas técnicas: segurança, escalabilidade, desempenho, confiabilidade e deployment
4. A proposta deve ser submetida por escrito (1 página) e aprovada pelo professor na entrega do Projeto 01
5. Deve incluir um plano de testes que valide as decisões arquiteturais

A proposta deve conter: título do projeto, problema a ser resolvido, padrões arquiteturais que serão aplicados, tecnologias previstas e critérios de sucesso mensurados.

4. POCs de Microsserviços Sugeridas

A seguir estão descritas 5 provas de conceito (POCs) que podem ser escolhidas pelas equipes. Cada POC possui escopo definido, padrões arquiteturais recomendados e decisões-chave a serem documentadas.

4.1. POC 1 — Real-Time (Salas/Rodadas + Sincronia de Evento)

Validar a capacidade de manter salas de jogo com comunicação bidirecional em tempo real via WebSocket, com latência inferior a 200ms (p95) e escala de até 1.000 jogadores simultâneos.

Escopo

- Motor de sala dedicado com WebSocket (ex.: Socket.IO, uWebSockets, Ably)
- Medição de latência end-to-end com percentis (p50, p95, p99)
- Simulação de carga progressiva: 100 → 500 → 1.000 jogadores
- Broadcast sincronizado: cronômetro regressivo + disparo simultâneo do desafio
- Medição de fan-out, reconexão e sincronização de estado

Padrões Recomendados	Load Balancing, Stateless/Stateful, Event Driven Architecture (EDA), Streaming vs Messaging, Bulkhead/Isolation Pattern, Rate Limit/Throttling
Decisões-Chave	Tecnologia de WebSocket, arquitetura de sala, limites de capacidade por nó

4.2. POC 2 — Antifraude Mínimo Viável

Construir um motor de risk scoring multifatorial capaz de detectar e colocar em quarentena jogadores suspeitos em tempo real.

Escopo

- Risk score multifatorial: device fingerprint + velocidade de ação + padrão de escolhas + correlação entre contas
- Quarentena automática baseada em threshold configurável
- Simulação de cenários: bot, multi-conta, conúio coordenado, multi-celular
- Integração com flags do Painel Admin

Padrões Recomendados	Circuit Breaker, CQRS, Event Sourcing, SAGA/Orchestration Pattern, Feature Flag, Anti-corruption Layer
Decisões-Chave	Modelo de scoring, regras de quarentena, integração com Painel Admin

4.3. POC 3 — Ledger + PIX Sandbox com Idempotência

Implementar um ledger de dupla entrada (double-entry) integrado com a API PIX em sandbox, garantindo idempotência e quarentena financeira.

Escopo

- Ledger de dupla entrada com tickets imutáveis
- Integração com API PIX em sandbox (Banco Central ou PSP homologado)
- Testes de idempotência: retry de saque não duplica crédito
- Quarentena financeira (Pending Balance) com liberação configurável
- Débito automático de API Fee biométrica (R$ 0,20–0,50) no fluxo de saque

Padrões Recomendados	ACID, Transactional Outbox, SAGA Pattern, Event Sourcing, Strong Consistency, Retry Pattern e DLQ, Idempotency
Decisões-Chave	Estrutura de ledger, fluxo de saque, política de quarentena

4.4. POC 4— Anti-Platform Throttling

Simular e medir estratégias de distribuição temporal de engajamento em plataformas externas, evitando throttling e penalizações algorítmicas.

Escopo

- Simulação de volume de engajamento direcionado em ambiente controlado
- Distribuição temporal com jitter variável e rate limiting por conteúdo
- Medição de thresholds estimados por plataforma (YouTube, Instagram, TikTok)
- Fila inteligente: balanceamento de carga entre campanhas ativas
- Pool de conteúdos rotativos com múltiplas URLs por anunciante

Padrões Recomendados	Rate Limit/Throttling, Load Balancing, Queues/PubSub/Fanout, Traffic Sharding, Bulkhead/Isolation, Circuit Breaker, Feature Flag
Decisões-Chave	Thresholds por plataforma, estratégia de distribuição, alertas e pausa automática

5. Critérios de Avaliação

A avaliação será composta por 5 dimensões, cada uma com peso específico:

#	Dimensão	O que será avaliado	Peso
1	**Arquitetura e Design**	Qualidade das decisões arquiteturais, uso correto dos padrões, ADRs, diagramas C4, trade-offs documentados.	**30%**
2	**Implementação e Código**	Qualidade do código, organização do repositório, testes, README, Docker/CI, boas práticas.	**25%**
3	**Videocast**	Clareza da apresentação, participação de todos, demonstração funcional, domínio técnico.	**20%**
4	**Testes e Validação**	Plano de testes, execução de testes de carga/resiliência/integração, métricas coletadas.	**15%**
5	**Originalidade e Profundidade**	Criatividade na solução, profundidade técnica, insights e lições aprendidas.	**10%**

Nota: cada integrante receberá avaliação individual baseada na participação na apresentação e nos commits do repositório.

Uso de IA — declaração obrigatória

O grupo deve incluir no README uma seção "Ferramentas de IA utilizadas" contendo:

- Quais ferramentas foram usadas (ex: GitHub Copilot, Cursor, ChatGPT, Claude, Gemini).
- Em quais partes do trabalho cada ferramenta atuou (geração de código, revisão, documentação, testes, análise, redação).
- Como a IA foi orientada — que tipo de instrução ou contexto foi fornecido.
- Uma avaliação honesta: o que funcionou, o que precisou ser corrigido e o que foi descartado.

> A omissão desta seção desclassifica a entrega, independentemente da qualidade técnica do restante do projeto

6. Tópicos Técnicos por Área

Os tópicos técnicos estão organizados em 5 áreas. Cada projeto deve abordar tópicos de pelo menos 2 dessas áreas. A área de Segurança foi expandida para incluir temas essenciais da engenharia de sistemas de larga escala na atualidade.

6.1. Segurança

A área de segurança abrange tanto tópicos do curso quanto temas contemporâneos essenciais para sistemas de larga escala. Em 2025–2026, a segurança de sistemas distribuídos evoluiu significativamente, com a adoção em massa de Zero Trust Architecture, segurança de supply chain de software, DevSecOps e preparação para criptografia pós-quântica.

Tópicos do Curso

- API e API Gateway Pattern (autenticação, autorização, rate limiting)
- Anti-corruption Layer (ACL)
- Service Mesh (mTLS, políticas de segurança entre serviços)
- Feature Flag / Feature Toggle (controle de exposição de funcionalidades)
- Zero Trust Architecture (ZTA): Modelo de segurança que elimina confiança implícita: cada requisição é verificada continuamente com micro-segmentação, MFA e validação de contexto do dispositivo.
- OAuth 2.0, OpenID Connect e JWT: Protocolos de autenticação e autorização para APIs e microsserviços: fluxos de token, escopos, refresh tokens e validação de claims.
- Secrets Management e Vaults: Gerenciamento seguro de credenciais, chaves de API e certificados com ferramentas como HashiCorp Vault, AWS Secrets Manager ou SOPS.
- Supply Chain Security (SBOM e Assinatura de Artefatos): Proteção da cadeia de suprimentos de software: geração de Software Bill of Materials, assinatura de imagens de container (Cosign/Sigstore), scan de vulnerabilidades em dependências.
- DevSecOps e Security Shift-Left: Integração de segurança no pipeline de CI/CD desde o início: SAST, DAST, SCA, container scanning e políticas como código (OPA/Rego).
- Criptografia em Trânsito e em Repouso (mTLS, TLS 1.3, Encryption at Rest): Proteção de dados com mutual TLS entre serviços, TLS 1.3 para comunicações externas e criptografia de dados armazenados com KMS.
- Segurança de Containers e Kubernetes (RBAC, Network Policies, Pod Security): Hardening de ambientes containerizados: RBAC granular, Network Policies, Pod Security Standards, image scanning e runtime protection.
- Proteção contra DDoS e Bot Mitigation: Estratégias de defesa contra ataques volumétricos e bots: WAF, rate limiting inteligente, CAPTCHA, challenge pages e fingerprinting.
- Observabilidade de Segurança (SIEM, Audit Logging, Threat Detection): Monitoramento contínuo de eventos de segurança: logs de auditoria imutáveis, correlação de eventos (SIEM), detecção de anomalias e alertas em tempo real.

- Criptografia Pós-Quântica (Post-Quantum Cryptography): Preparação para a era quântica: algoritmos resistentes a computadores quânticos (CRYSTALS-Kyber, CRYSTALS-Dilithium), crypto-agility e inventário criptográfico.

6.2. Escalabilidade

- Load Balancing Pattern
- Multi-tenant e Single Tenant app
- Database Sharding Pattern
- Consistent Hashing
- Técnicas de Sharding avançada (Shuffle Sharding)
- Queues, PubSub e Messaging Fanout Pattern
- Arquitetura com Containers
- Arquitetura utilizando Serverless
- Stateless, Stateful e Imutabilidade
- Service Registry e Discovery Pattern
- Traffic Sharding
- Backends for Frontends (BFF)
- Aggregator Design Pattern / API Composition Pattern
- Monolito vs Microsserviços
- Monolito vs Monolito Modular

6.3. Desempenho

- Caching de dados e de objetos
- Cache-Aside Pattern
- SYNC vs ASYNC, Arquitetura baseada em Eventos/Mensagens
- Streaming vs Messaging
- CQRS (Command Query Responsibility Segregation)
- Normalização e Desnormalização / Materialized View
- Polyglot Persistence Patterns
- Database-per-service e Shared-database-per-service
- Request Hedging Pattern
- Rate Limit / Throttling Pattern
- Load Shedding

6.4. Confiabilidade

- Circuit Breaker
- Retry Pattern e DLQ
- Bulkhead / Isolation Pattern
- SAGA Pattern / Orchestration Pattern
- Choreography Pattern
- Event Sourcing
- Event Driven Architectures (EDA)
- Change Data Capture (CDC) Pattern
- Transactional Outbox Pattern
- Strong vs Eventual Consistency
- ACID
- Sidecar, Ambassador e Adapter Pattern
- Service Mesh
- Strangler Fig Pattern
- Anti-corruption Layer (ACL)
- Tight Coupling and Loose Coupling

6.5. Deployment

- Estratégia de Deployment (como definir)
- In-place vs Immutable Deployment / Agent vs Agentless
- Esteira Pipeline, Build e Deployment
- All at Once / Big Bang / Full Deployment
- Rolling / Incremental / Gradual Deployment
- Blue-Green Deployment
- Canary Release Deployment
- Shadow Deployment / Dark Launch / Shadow Testing
- Feature Flag / Feature Toggle

7. Checklist de Entrega Final

Use este checklist para verificar se todas as entregas do Projeto 03 estão completas:

✔	Item de Entrega
[ ]	Repositório Git criado e acessível (público ou com acesso ao professor)
[ ]	README.md com descrição do projeto, instruções de execução e link do videocast
[ ]	Diagrama de arquitetura C4 (níveis 1 e 2, no mínimo)
[ ]	ADRs (Architecture Decision Records) completos e documentados
[ ]	Código-fonte funcional com testes
[ ]	Docker Compose ou equivalente para execução local
[ ]	Videocast gravado (15–30 min) com participação de todos os integrantes
[ ]	Link do videocast incluído no README
[ ]	Histórico de commits mostrando contribuição de todos os membros
[ ]	Plano e resultados de testes (carga, resiliência, integração)
[ ]	Trade-offs documentados com justificativas

8. Observações Finais

- As equipes que escolherem POCs sugeridas podem adaptá-las, adicionando ou removendo itens do escopo com justificativa documentada.
- O uso de ferramentas de observabilidade (Prometheus, Grafana, Jaeger, etc.) será considerado um diferencial.
- Projetos que implementarem estratégias de deployment (CI/CD pipeline, Blue-Green, Canary) receberão pontuação adicional na dimensão de Originalidade.
- Projetos que abordarem temas expandidos de segurança (Zero Trust, DevSecOps, Supply Chain Security) terão valorização especial.
- Plágio ou reutilização de código sem atribuição resultará em nota zero para toda a equipe.
- Dúvidas devem ser encaminhadas ao professor com antecedência mínima de 48 horas antes da data de entrega.

Bom projeto a todos!